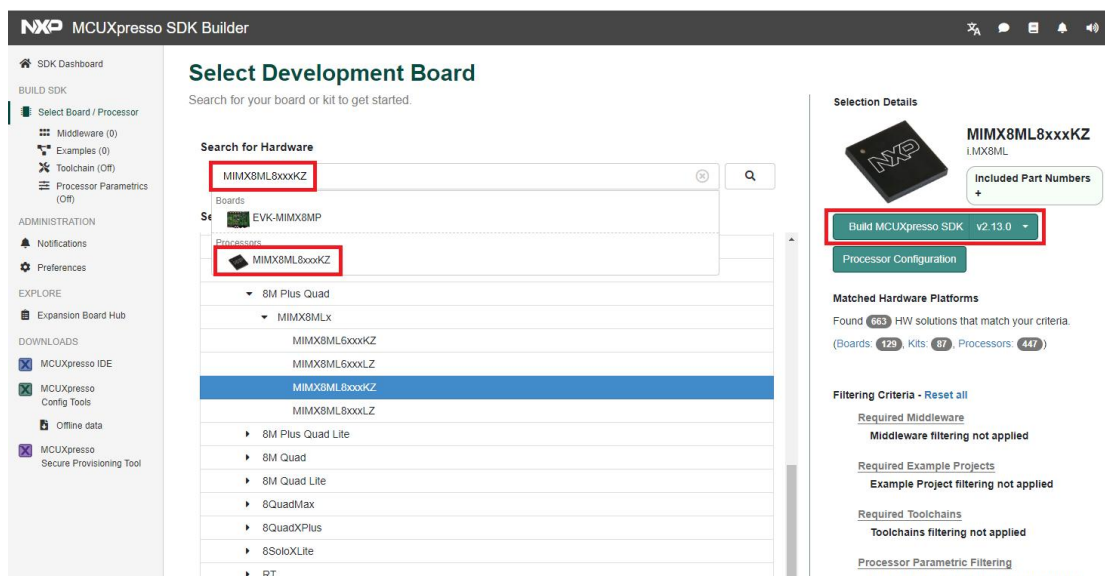


A53 and M7 Parallel Approach on DEBIX

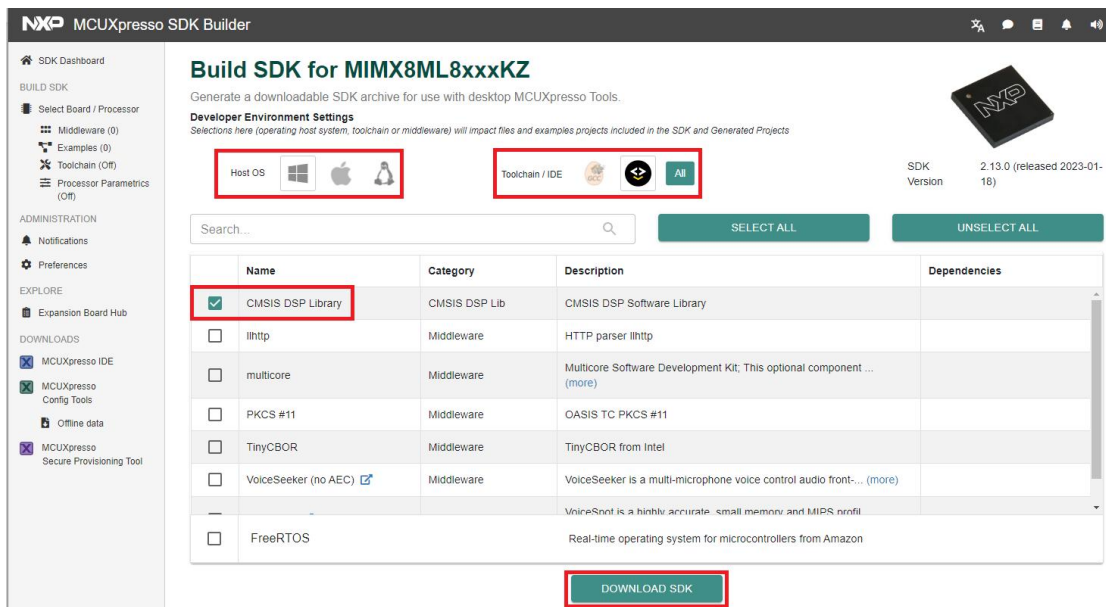
1. Run M7 core alone

1.1. Download firmware

1. Get the SDK package from NXP official website: <https://mcuxpresso.nxp.com/en/select>.
(the package named EVK-MIMX8MP-hello_world.zip)
- Type **MIMX8ML8xxxKZ** in the search box of “Search for Hardware”
- Select **MIMX8ML8xxxKZ** processor
- Click **Build MCUXpresso SDK** button on the right side



- Select the Host OS, and select Toolchain / IDE
- Check **CMSIS DSP Library**, and click **DOWNLOAD SDK**.



Build SDK for MIMX8ML8xxxKZ

Generate a downloadable SDK archive for use with desktop MCUXpresso Tools.

Developer Environment Settings
 Selections here (operating host system, toolchain or middleware) will impact files and examples projects included in the SDK and Generated Projects

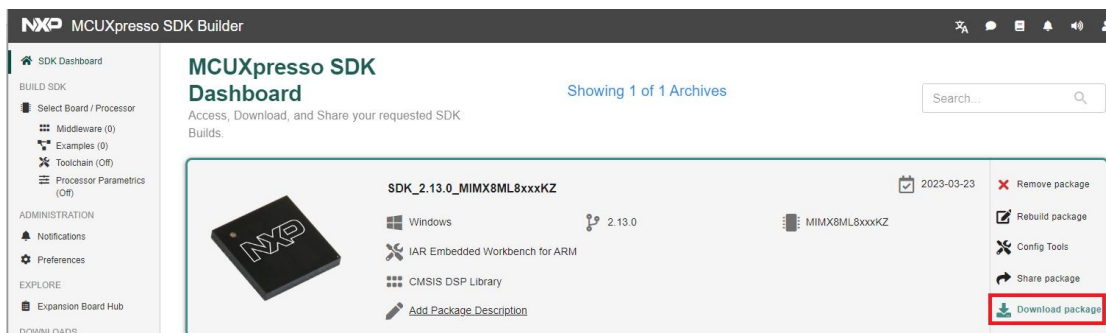
Host OS: Windows, macOS, Linux
 Toolchain / IDE: IAR, GCC, All

SDK Version: 2.13.0 (released 2023-01-16)

	Name	Category	Description	Dependencies
☒	CMSIS DSP Library	CMSIS DSP Lib	CMSIS DSP Software Library	
☐	ilhttp	Middleware	HTTP parser ilhttp	
☐	multicore	Middleware	Multicore Software Development Kit. This optional component ... (more)	
☐	PKCS #11	Middleware	OASIS TC PKCS #11	
☐	TinyCBOR	Middleware	TinyCBOR from Intel	
☐	VoiceSeeker (no AEC)	Middleware	VoiceSeeker is a multi-microphone voice control audio front-... (more)	
☐	VoiceSpot	Middleware	VoiceSpot is a highly accurate, small memory and MIPS profile	
☐	FreeRTOS		Real-time operating system for microcontrollers from Amazon	

DOWNLOAD SDK

- Select Download package



MCUXpresso SDK Dashboard

Showing 1 of 1 Archives

Search...

SDK_2.13.0_MIMX8ML8xxxKZ

2023-03-23

Remove package

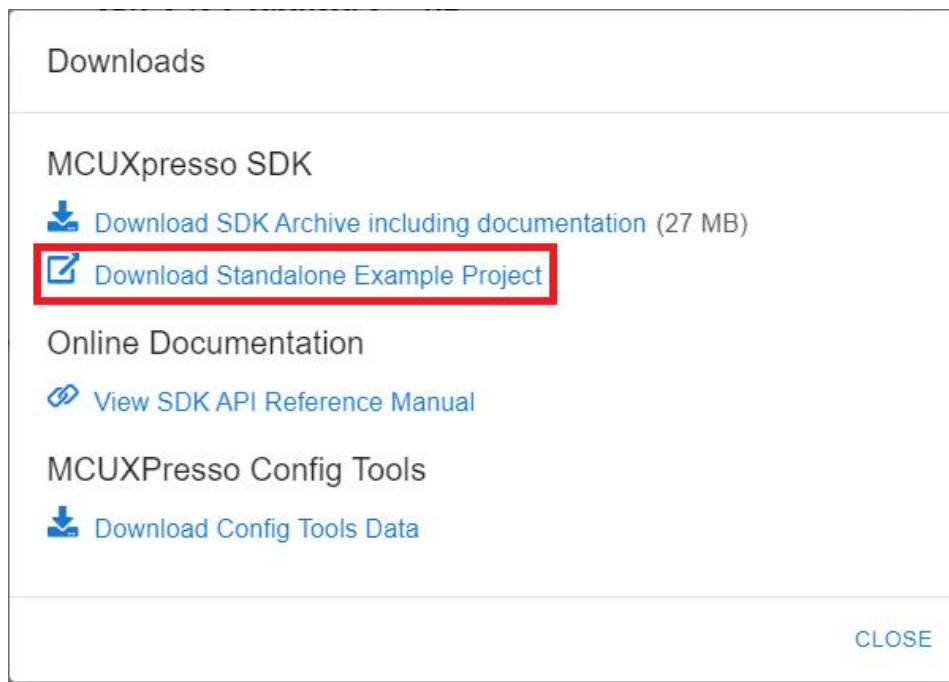
Rebuild package

Config Tools

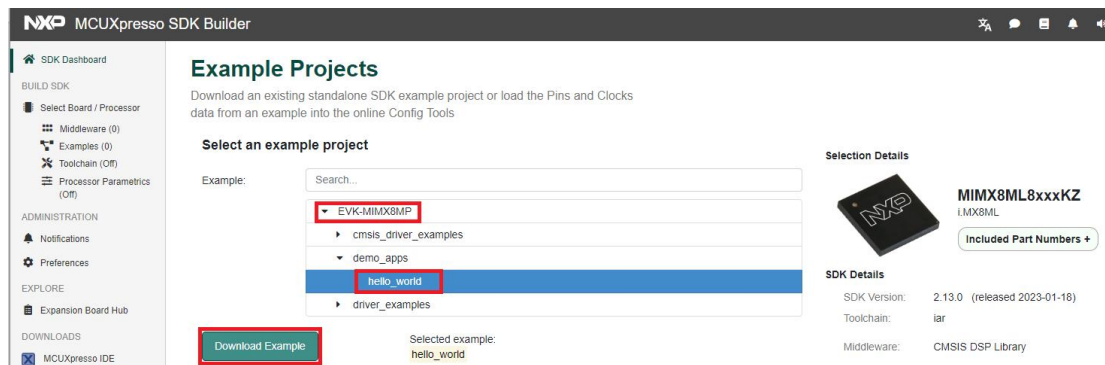
Share package

Download package

- Select Download Standalone Example Project



- Select `hello_world`, and click **Download Example** button



After downloading, you will get a `EVK-MIMX8MP-hello_world.zip`, which is the project you will use.

2. Unzip it and open it with IAR

1) Add simple delay 1S function

```
void delay_1s()
{
    unsigned int i = 0;
    unsigned int j = 0;
    for(i = 0; i < 10000; i++)
        for(j = 0; j < 20000; j++);
}
```

}

- 2) Add the following code to while(1) in main function:

```
delay_1s();
```

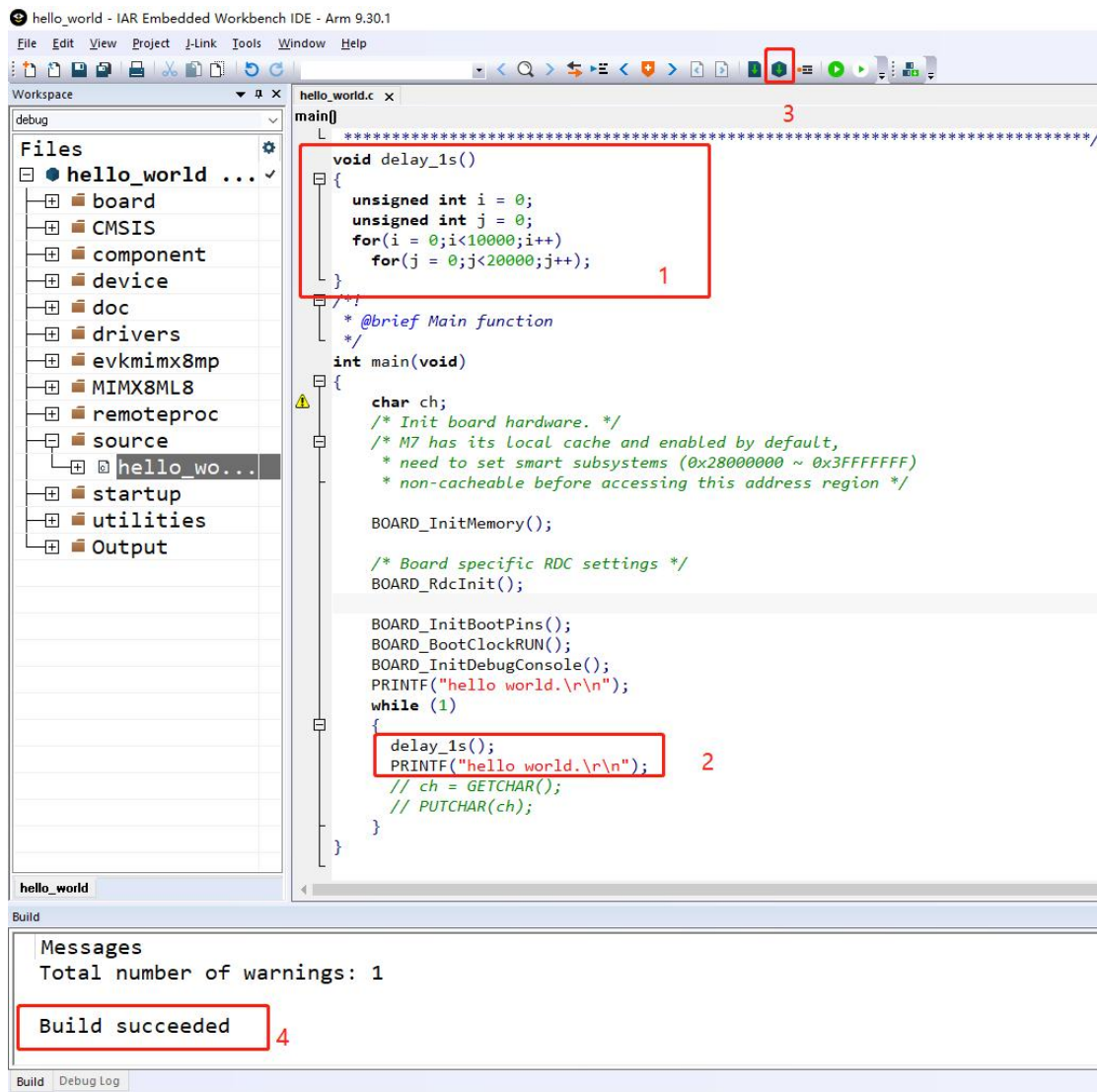
```
PRINTF("hello world.\r\n");
```

- 3) Comment out the original code:

```
// ch = GETCHAR();
```

```
// PUTCHAR(ch);
```

- 4) Click compile button



1.2. Connect Hardware

Note: The original serial port 2 is used as debug port to print boot log, M7 core and A53 core share system resources, so the M7 print information will be output from serial port 4.

- Connect serial port 2 and serial port 4 to the computer via USB-TTL.

1.3. Boot M7 core alone and verify the hello_world.bin file

- Execute the following command on DEBIX:

```
sudo mount /dev/mmcblk1p1 /boot
```

```
sudo cp hello_world.bin /boot
```

```
sudo umount /boot
```

```
debix@imx8mpevk:~$ ls
Desktop    Image      Public      gstshark_2023-02-24_02:50:01
Documents  Music      Templates   hello_world.bin
Downloads  Pictures   Videos
debix@imx8mpevk:~$ sudo mount /dev/mmcblk1p1 /boot
[sudo] password for debix:
mount: /boot: /dev/mmcblk1p1 already mounted on /boot.
debix@imx8mpevk:~$ sudo cp hello_world.bin /boot
debix@imx8mpevk:~$ sudo umount /boot
debix@imx8mpevk:~$
```

- Reboot DEBIX
- Enter the uboot command line and run the command to start the M7 core:

```
fatload mmc 1:1 0x48000000 hello_world.bin;cp.b 0x48000000 0x7e0000 20000;
```

```
bootaux 0x7e0000
```

```
u-boot=>
u-boot=> fatload mmc 1:1 0x48000000 hello_world.bin;cp.b 0x48000000 0x7e0000 20000;
12688 bytes read in 3 ms (4 MiB/s)
u-boot=> bootaux 0x7e0000
## Starting auxiliary core stack = 0x20020000, pc = 0x00002955...
u-boot=>
```

At this point, UART4 prints `hello world` in the serial terminal, which proves that the M7 core can work normally.

[illegible]

2. Parallel A53 and M7 cores

2.1. Modify kernel source code

1. Modify the device tree `/arch/arm64/boot/dts/freescale/imx8mp-evk.dts`

Add the following code under the root node:

```
reserved-memory {
    #address-cells = <2>;
    #size-cells = <2>;
    ranges;

    m4_reserved: m4@0x80000000 {
        no-map;
        reg = <0 0x80000000 0 0x1000000>;
    };

    vdev0vring0: vdev0vring0@55000000 {
        reg = <0 0x55000000 0 0x8000>;
        no-map;
    };

    vdev0vring1: vdev0vring1@55008000 {
        reg = <0 0x55008000 0 0x8000>;
        no-map;
    };

    vdevbuffer: vdevbuffer@55400000 {
        compatible = "shared-dma-pool";
        reg = <0 0x55400000 0 0x100000>;
        no-map;
    };

    rsc_table: rsc_table@550ff000 {
```

```

        reg = <0 0x550ff000 0 0x1000>;
        no-map;
    };
};
imx8mp-cm7 {
    compatible = "fsl,imx8mp-cm7";
    rsc-da = <0x55000000>;
    clocks = <&clk IMX8MP_CLK_M7_DIV>;
    mbox-names = "tx", "rx", "rxd";
    mboxes = <&mu 0 1
        &mu 1 1
        &mu 3 1>;
    memory-region = <&vdevbuffer>, <&vdev0vring0>, <&vdev0vring1>, <&rsc_table>;
    status = "okay";
};

```

2. Modify the M7 core device driver /drivers/remoteproc/imx_rproc.c

- line 28, add the following code:

```

#define TCML_ADDR_IMX8M          0x7e0000
#define TCML_ADDR_IMX8_CM4_1    0x34fe0000
#define TCML_ADDR_IMX8_CM4_2    0x38fe0000

26 #include "remoteproc_internal.h"
27
28 #define TCML_ADDR_IMX8M          0x7e0000
29 #define TCML_ADDR_IMX8_CM4_1    0x34fe0000
30 #define TCML_ADDR_IMX8_CM4_2    0x38fe0000

```

- line 139, add the following code:

```

u32      m_core_ddr_addr;
u32      last_load_addr;
u32      m4_start_addr;

137 struct device      **pm_devices;
138 struct device_link  **pm_devices_link;
139 u32      m_core_ddr_addr;
140 u32      last_load_addr;
141 u32      m4_start_addr;
142 };
143
144 static struct imx_sc_ipc *ipc_handle;

```

- line 146, add the following code:

```
static const struct imx_rproc_att imx_rproc_att_imx8mp[] = {  
    /* dev addr , sys addr , size , flags */  
    /* ITCM */  
    {0x00000000, 0x007E0000, 0x00020000, ATT_OWN},  
    /* OCRAM_S */  
    {0x00180000, 0x00180000, 0x00009000, 0},  
    /* OCRAM */  
    {0x00900000, 0x00900000, 0x00090000, 0},  
    /* QSPI Code - alias */  
    {0x08000000, 0x08000000, 0x08000000, 0},  
    /* DDR (Code) - alias */  
    {0x10000000, 0x40000000, 0xFFE0000, 0},  
    /* DTCM */  
    {0x20000000, 0x00800000, 0x00020000, ATT_OWN},  
    /* OCRAM_S - alias */  
    {0x20180000, 0x00180000, 0x00008000, ATT_OWN},  
    /* OCRAM */  
    {0x20200000, 0x00900000, 0x00090000, ATT_OWN},  
    /* DDR (Data) */  
    {0x40000000, 0x40000000, 0x80000000, 0},  
};
```

```
144 static struct imx_sc_ipc *ipc_handle;  
145  
146 static const struct imx_rproc_att imx_rproc_att_imx8mp[] = {  
147     /* dev addr , sys addr , size , flags */  
148     /* ITCM */  
149     {0x00000000, 0x007E0000, 0x00020000, ATT_OWN},  
150     /* OCRAM_S */  
151     {0x00180000, 0x00180000, 0x00009000, 0},  
152     /* OCRAM */  
153     {0x00900000, 0x00900000, 0x00090000, 0},  
154     /* QSPI Code - alias */  
155     {0x08000000, 0x08000000, 0x08000000, 0},  
156     /* DDR (Code) - alias */  
157     {0x10000000, 0x40000000, 0xFFE0000, 0},  
158     /* DTCM */  
159     {0x20000000, 0x00800000, 0x00020000, ATT_OWN},  
160     /* OCRAM_S - alias */  
161     {0x20180000, 0x00180000, 0x00008000, ATT_OWN},  
162     /* OCRAM */  
163     {0x20200000, 0x00900000, 0x00090000, ATT_OWN},  
164     /* DDR (Data) */  
165     {0x40000000, 0x40000000, 0x80000000, 0},  
166 };
```

- line 317, add the following code:

```
static const struct imx_rproc_dcfg imx_rproc_cfg_imx8mp = {
    .att = imx_rproc_att_imx8mp,
    .att_size = ARRAY_SIZE(imx_rproc_att_imx8mp),
    .elf_mem_hook = true,
    .method = IMX_ARM_SMCCC,
};

316 };
317 static const struct imx_rproc_dcfg imx_rproc_cfg_imx8mp = {
318     .att = imx_rproc_att_imx8mp,
319     .att_size = ARRAY_SIZE(imx_rproc_att_imx8mp),
320     .elf_mem_hook = true,
321     .method = IMX_ARM_SMCCC,
322 };
323
324 static const struct imx_rproc_dcfg imx_rproc_cfg_imx8mq = {
```

- line 461, add the following code:

```
/*
"m4 always start up from TCM. The SCU will copy the first 32 bytes of the binary to TCM
if the start address is not TCM. The TCM region [0x1FFE0000-0x1FFE001F] is reserved for this purpose."
Therefore:
For imx8q and imx8x, it is not necessary to copy the stack pointer and reset vector from ddr to tcm.
Instead, determine if firmware was loaded into TCM or DDR and provide correct start address to SCU.
Like 8M family, DDR4 address is defined in device tree node m4_reserved, m7_reserved, or mcore_reserved
*/
static void imx8_set_start_addr(struct rproc *rproc, u32 addr_tcm) {
    struct imx_rproc *priv = rproc->priv;

    if(priv->m_core_ddr_addr && priv->last_load_addr >= priv->m_core_ddr_addr) {
        priv->m4_start_addr = priv->m_core_ddr_addr;
        dev_info(priv->dev, "Setting Cortex M4 start address to DDR 0x%08x\n", priv->m4_start_addr);
    } else {
        priv->m4_start_addr = addr_tcm;
        dev_info(priv->dev, "Setting Cortex M4 start address to TCM 0x%08x\n", priv->m4_start_addr);
    }
}

/*
Stack pointer and reset vector must be initialized
*/
static void imx_8m_setup_stack(struct rproc *rproc)
{
```

```

struct imx_rproc *priv = rproc->priv;
void __iomem *io_tcml = ioremap(TCML_ADDR_IMX8M, 8);
//initialize tcml stack pointer and reset vector
if(priv->m_core_ddr_addr && priv->last_load_addr >= priv->m_core_ddr_addr) {
    void __iomem *io_ddr = ioremap(priv->m_core_ddr_addr, 8);
    dev_info(priv->dev, "Setting up stack pointer and reset vector from firmware in DDR\n");
    writel(readl(io_ddr), io_tcml);
    writel(readl(io_ddr + 4), io_tcml + 4);
    iounmap(io_ddr);
} else {
    dev_info(priv->dev, "Setting up stack pointer and reset vector from firmware in TCML\n");
    writel(readl(io_tcml), io_tcml);
    writel(readl(io_tcml + 4), io_tcml + 4);
}
dev_info(priv->dev, "Stack: 0x%x\n", readl(io_tcml));
dev_info(priv->dev, "Reset Vector: 0x%x\n", readl(io_tcml + 4));
iounmap(io_tcml);
}

```

```

456     priv->tx_ch = NULL;
457     priv->rx_ch = NULL;
458     priv->rxdb_ch = NULL;
459     priv->txdb_ch = NULL;
460 }
461 /*
462 "M4 always start up from TCM. The SCU will copy the first 32 bytes of the binary to TCM
463 if the start address is not TCM. The TCM region [0x1FFE0000-0x1FFE001F] is reserved for this purpose."
464 Therefore:
465 For imx8q and imx8x, it is not necessary to copy the stack pointer and reset vector from ddr to tcm.

```

- line 518 and 523, both add the following code:

```

imx_8m_setup_stack(rproc);

```

```

516     switch (dcfg->method) {
517     case IMX_DIRECT_MMIO:
518         imx_8m_setup_stack(rproc);
519         ret = regmap_update_bits(priv->regmap, dcfg->src_reg,
520                                 dcfg->src_mask, dcfg->src_start);
521         break;
522     case IMX_ARM_SMCCC:
523         imx_8m_setup_stack(rproc);
524         arm_smccc_smc(IMX_SIP_SRC, IMX_SIP_SRC_M4_START, 0, 0, 0, 0, 0, 0, &res);
525         ret = res.a0;
526         break;

```

- Comment out code from lines 531-541:

```

// if (priv->id == 1)
//     ret = imx_sc_pm_cpu_start(ipc_handle, priv->rsrc, true, 0x38fe0000);
// else if (!priv->id)
//     ret = imx_sc_pm_cpu_start(ipc_handle, priv->rsrc, true, 0x34fe0000);
// else

```

- line 542, add the following code:

```
if (priv->id == 1) {
    imx8_set_start_addr(rproc, TCML_ADDR_IMX8_CM4_2);
    ret = imx_sc_pm_cpu_start(ipc_handle, priv->rsrc, true, priv->m4_start_addr);
} else if (!priv->id) {
    imx8_set_start_addr(rproc, TCML_ADDR_IMX8_CM4_1);
    ret = imx_sc_pm_cpu_start(ipc_handle, priv->rsrc, true, priv->m4_start_addr);
} else
```

```
534         return imx_rproc_ready(rproc);
535     }
536
537     // if (priv->id == 1)
538     //     ret = imx_sc_pm_cpu_start(ipc_handle, priv->rsrc, true, 0x38fe0000);
539     // else if (!priv->id)
540     //     ret = imx_sc_pm_cpu_start(ipc_handle, priv->rsrc, true, 0x34fe0000);
541     // else
542     if (priv->id == 1) {
543         imx8_set_start_addr(rproc, TCML_ADDR_IMX8_CM4_2);
544         ret = imx_sc_pm_cpu_start(ipc_handle, priv->rsrc, true, priv->m4_start_addr);
545     } else if (!priv->id) {
546         imx8_set_start_addr(rproc, TCML_ADDR_IMX8_CM4_1);
547         ret = imx_sc_pm_cpu_start(ipc_handle, priv->rsrc, true, priv->m4_start_addr);
548     } else
549         ret = -EINVAL;
550     break;
551     case IMX_IPC_ONLY:
552         return -ENOTSUPP;
```

- line 607 and 609, both change to the following code:

```
ret = imx_sc_pm_cpu_start(ipc_handle, priv->rsrc, false, priv->m4_start_addr);
```

```
604         break;
605     case IMX_SCU_API:
606         if (priv->id == 1)
607             ret = imx_sc_pm_cpu_start(ipc_handle, priv->rsrc, false, priv->m4_start_addr);
608         else if (!priv->id)
609             ret = imx_sc_pm_cpu_start(ipc_handle, priv->rsrc, false, priv->m4_start_addr);
610         else
611             ret = -EINVAL;
612         break;
```

- line 771, add the following code:

```
//get m4/m7 ddr address from device tree
if(0 == strcmp(it.node->name, "m4") || 0 == strcmp(it.node->name, "m7")
    || 0 == strcmp(it.node->name, "m_core")) {
    priv->m_core_ddr_addr = rmem->base;
    dev_info(priv->dev, "%s ddr @ 0x%x\n", it.node->name, (u32) rmem->base);
}
```

```

770         return -ENOMEM;
771 //get m4/m7 ddr address from device tree
772         if(0 == strcmp(it.node->name, "m4") || 0 == strcmp(it.node->name, "m7")
773 || 0 == strcmp(it.node->name, "m_core")) {
774             priv->m_core_ddr_addr = rmem->base;
775             dev_info(priv->dev, "%s ddr @ 0x%x\n", it.node->name, (u32) rmem->base);
776         }
777
778         rproc_add_carveout(rproc, mem);
779         index++;
780     }
781
782     return 0;
783 }

```

- line 824, add the following code:

```

static u64 imx_rproc_elf_get_boot_addr(struct rproc *rproc,
                                     const struct firmware *fw)
{
    struct imx_rproc *priv = rproc->priv;
    if (!priv->early_boot) {
        //save the location of the last firmware load for start function
        priv->last_load_addr = rproc_elf_get_boot_addr(rproc, fw);
        return (u64)priv->last_load_addr;
    }
    return 0;
}

```

```

822     spin_unlock_irqrestore(&priv->mu_lock, flags);
823 }
824 static u64 imx_rproc_elf_get_boot_addr(struct rproc *rproc,
825                                     const struct firmware *fw)
826 {
827     struct imx_rproc *priv = rproc->priv;
828
829
830     if (!priv->early_boot) {
831         //save the location of the last firmware load for start function
832         priv->last_load_addr = rproc_elf_get_boot_addr(rproc, fw);
833         return (u64)priv->last_load_addr;
834     }
835
836     return 0;
837 }
838

```

- line 904, change to the following code:

```

_get_boot_addr = imx_rproc_elf_get_boot_addr,

```

```
902     .sanity_check    = rproc_elf_sanity_check,  
903     .get_boot_addr   = imx_rproc_elf_get_boot_addr,  
904     // .get_boot_addr = rproc_elf_get_boot_addr,  
905 };  
906
```

- line 1101, add the following code:

```
static int imx_rproc_parse_dt(struct device *dev, struct imx_rproc *priv)  
{  
    int ret = 0;  
  
    if (priv->dcfg->method == IMX_SCU_API) {  
        ret = of_property_read_u32(dev->of_node, "core-index", &priv->id);  
        if (ret)  
            dev_err(dev, "No reg <core index id>\n");  
    }  
    return ret;  
}
```

```
1099     return 0;  
1100 }  
1101 static int imx_rproc_parse_dt(struct device *dev, struct imx_rproc *priv)  
1102 {  
1103     int ret = 0;  
1104  
1105     if (priv->dcfg->method == IMX_SCU_API) {  
1106         ret = of_property_read_u32(dev->of_node, "core-index", &priv->id);  
1107         if (ret)  
1108             dev_err(dev, "No reg <core index id>\n");  
1109     }  
1110  
1111     return ret;  
1112 }  
1113  
1114 static int imx_rproc_detect_mode(struct imx_rproc *priv)
```

- Comment out code from lines 1148-1152:

```
// ret = of_property_read_u32(dev->of_node, "core-index", &priv->id);  
// if (ret) {  
//     dev_err(dev, "No reg <core index id>\n");  
//     return ret;  
// }
```

- line 1271, add the following code:

```
priv->m_core_ddr_addr = 0;  
priv->last_load_addr = 0;
```

```
1267     priv = rproc->priv;
1268     priv->rproc = rproc;
1269     priv->dcfg = dcfg;
1270     priv->dev = dev;
1271     priv->m_core_dds_addr = 0;
1272     priv->last_load_addr = 0;
```

- line 1293, add the following code:

```
ret = imx_rproc_parse_dt(dev, priv);
if (ret)
    goto err_put_mbox;
```

```
1291     /* mbox is optional, so not fail here */
1292 }
1293 ret = imx_rproc_parse_dt(dev, priv);
1294 if (ret)
1295     goto err_put_mbox;
1296
1297 ret = imx_rproc_addr_init(priv, pdev);
```

- line 1356, add the following code:

```
if (!IS_ERR(priv->txdb_ch))
    mbox_free_channel(priv->txdb_ch);
if (!IS_ERR(priv->rxdb_ch))
    mbox_free_channel(priv->rxdb_ch);
if (!IS_ERR(priv->tx_ch))
    mbox_free_channel(priv->tx_ch);
if (!IS_ERR(priv->rx_ch))
    mbox_free_channel(priv->rx_ch);
```

```

1352 ~ static int imx_rproc_remove(struct platform_device *pdev)
1353 {
1354     struct rproc *rproc = platform_get_drvdata(pdev);
1355     struct imx_rproc *priv = rproc->priv;
1356     if (!IS_ERR(priv->txdb_ch))
1357         mbox_free_channel(priv->txdb_ch);
1358     if (!IS_ERR(priv->rxdb_ch))
1359         mbox_free_channel(priv->rxdb_ch);
1360     if (!IS_ERR(priv->tx_ch))
1361         mbox_free_channel(priv->tx_ch);
1362     if (!IS_ERR(priv->rx_ch))
1363         mbox_free_channel(priv->rx_ch);
1364     if (!priv->early_boot)
1365         clk_disable_unprepare(priv->clk);
1366     rproc_del(rproc);

```

- line 1379, change to the following code:

```

{ .compatible = "fsl,imx8mp-cm7", .data = &imx_rproc_cfg_imx8mp },
{ .compatible = "fsl,imx8mp-cm4", .data = &imx_rproc_cfg_imx8mq },
1377 { .compatible = "fsl,imx8mm-cm4", .data = &imx_rproc_cfg_imx8mq },
1378 { .compatible = "fsl,imx8mn-cm7", .data = &imx_rproc_cfg_imx8mn },
1379 //{ .compatible = "fsl,imx8mp-cm7", .data = &imx_rproc_cfg_imx8mn },
1380 { .compatible = "fsl,imx8mp-cm7", .data = &imx_rproc_cfg_imx8mp },
1381 { .compatible = "fsl,imx8qxp-cm4", .data = &imx_rproc_cfg_imx8qxp },

```

3. Modify the M7 core device driver /drivers/remoteproc/remoteproc_core.c

- line 1113, add the following code:

```

if ((int)rproc->table_ptr->num < 0) {
    WARN_ON(rproc->table_ptr->num);
    rproc->table_ptr->num=0;
    return 0;
}

1110 struct device *dev = &rproc->dev;
1111 rproc_handle_resource_t handler;
1112 int ret = 0, i;
1113 if ((int)rproc->table_ptr->num < 0) {
1114     WARN_ON(rproc->table_ptr->num);
1115     rproc->table_ptr->num=0;
1116     return 0;
1117 }

```

- line 1382, change to the following code:

```

if (loaded_table && rproc->cached_table) {

```

```

1379
1380     loaded_table = rproc_find_loaded_rsc_table(rproc, fw);
1381     //if (loaded_table) {
1382     if (loaded_table && rproc->cached_table) {
1383         memcpy(loaded_table, rproc->cached_table, rproc->table_sz);
1384         rproc->table_ptr = loaded_table;
1385     }

```

4. Modify M7 core device driver /drivers/remoteproc/remoteproc_elf_loader.c

- line 136, comment out the original code and modify it to the following code:

```

//rproc->ops->elf_memcpy(rproc, dest, src, count);
else
    rproc->ops->elf_memcpy(rproc, dest, src, count);

```

- line 147, comment out the original code and modify it to the following code:

```

//rproc->ops->elf_memset(rproc, s, c, count);
else
    rproc->ops->elf_memset(rproc, s, c, count);

```

```

132 static void rproc_elf_memcpy(struct rproc *rproc, void *dest, const void *src, size_t count)
133 {
134     if (!rproc->ops->elf_memcpy)
135         memcpy(dest, src, count);
136     else
137         rproc->ops->elf_memcpy(rproc, dest, src, count);
138
139     //rproc->ops->elf_memcpy(rproc, dest, src, count);
140
141 }
142
143 static void rproc_elf_memset(struct rproc *rproc, void *s, int c, size_t count)
144 {
145     if (!rproc->ops->elf_memset)
146         memset(s, c, count);
147     else
148         rproc->ops->elf_memset(rproc, s, c, count);
149     //rproc->ops->elf_memset(rproc, s, c, count);
150 }

```

2.2. Recompile the kernel

- Copy the compiled Image file and the hello_world.bin file running on the M7 core to the /boot directory of DEBIX. Execute the following command:

```
sudo mount /dev/mmcblk1p1 /boot
```

```
sudo cp Image hello_world.bin /boot
```

```
sudo umount /boot
```

```
sudo reboot
```

```
debix@imx8mpevk:~$ ls
Desktop  Image      Public      gstshark_2023-02-24_02:50:01
Documents Music      Templates   hello_world.bin
Downloads Pictures  Videos
debix@imx8mpevk:~$ sudo mount /dev/mmcblk1p1 /boot
debix@imx8mpevk:~$ sudo cp Image hello_world.bin /boot
debix@imx8mpevk:~$ sudo umount /boot
debix@imx8mpevk:~$ sudo reboot
```

2.3. Boot A53 core

- `sudo reboot`
- Press any key during the uboot boot to enter uboot command line mode.

```
fatload mmc 1:1 0x48000000 hello_world.bin;cp.b 0x48000000 0x7e0000 2000;
```

```
bootaux 0x7e0000
```

```
Hit any key to stop autoboot: 0
u-boot=>
u-boot=>
u-boot=>
u-boot=> fatload mmc 1:1 0x48000000 hello_world.bin;cp.b 0x48000000 0x7e0000 2000;
12688 bytes read in 3 ms (4 MiB/s)
u-boot=> bootaux 0x7e0000
## Starting auxiliary core stack = 0x20020000, pc = 0x00002955...
```

- Execute `boot` command to boot the A53 core.

```
u-boot=> boot
starting USB...
```

The serial port 2 (debug serial port) prints the kernel boot log normally and successfully boots Linux as follows:

```
[ OK ] Started Snap Daemon.
      Starting Wait until snapd is fully seeded...
      Starting Time & Date Service...
[ OK ] Started Time & Date Service.
[ OK ] Finished Wait until snapd is fully seeded.

Ubuntu 20.04.3 LTS imx8mpevk ttyMX1

imx8mpevk login:
imx8mpevk login: 
```

Serial port 4 (M7 core output) prints normally at 1S interval:

```
hello world.  
hello world.  
hello world.  
|
```

If you enter the system normally, it means that the A53 core and M7 are working at the same time.

3. Develop M7 firmware on debix

3.1. Get the SDK source code and copy it to debix

Follow the steps in section 1.1. of "Run M7 core alone" to download the SDK at the website:

<https://mcuxpresso.nxp.com/en/example-projects?uvid=493518>

 EVK-MIMX8MP-hello_world.zip

2023/3/23 15:14

3.2. Configure the development environment

1. Install cmake

- Run `sudo apt install cmake` command

```
debix@imx8mpevk:~/hello_world$ sudo apt install cmake  
[sudo] password for debix:  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
The following additional packages will be installed:
```

- Replace the dynamic link library

```
sudo rm /usr/lib/libcurl.so.4 // remove the original dynamic library
```

```
sudo ln -s /usr/lib/libcurl.so.4.7.0 /usr/local/lib/libcurl.so.4
```

```
ls -l /usr/local/lib/libcurl.so.4 //check whether the link is successful
```

```
debix@imx8mpevk:~/hello_world$ sudo rm /usr/lib/libcurl.so.4  
debix@imx8mpevk:~/hello_world$ sudo ln -s /usr/lib/libcurl.so.4.7.0 /usr/local/lib/libcurl.so.4  
debix@imx8mpevk:~/hello_world$ ls -l /usr/local/lib/libcurl.so.4  
lrwxrwxrwx 1 root root 25 Mar 10 04:00 /usr/local/lib/libcurl.so.4 -> /usr/lib/libcurl.so.4.7.0
```

2. Install the compiler

- Download the compiler

```
wget
```

```
https://armkeil.blob.core.windows.net/developer/Files/downloads/gnu-rm/9-2019q4/gcc-arm-  
none-eabi-9-2019-q4-major-aarch64-linux.tar.bz2
```

- Unpack the compiler and configure the environment variables

```
tar -xvf gcc-arm-none-eabi-9-2019-q4-major-aarch64-linux.tar.bz2
```

```
debix@imx8mpevk:~/tools$ tar -xvf gcc-arm-none-eabi-9-2019-q4-major-aarch64-linu  
x.tar.bz2
```

```
sudo vi ~/.bashrc
```

```
export ARMGCC_DIR=~/.tools/gcc-arm-none-eabi-9-2019-q4-major
```

```
fi
```

```
export ARMGCC_DIR=~/.tools/gcc-arm-none-eabi-9-2019-q4-major
```

Make the environment variables effective via `source ~/.bashrc` command.

3.3. Compile the firmware

- Enter the project directory and execute the compiler script `./build_all.sh`

```
debix@imx8mpevk:~$ cd hello_world  
debix@imx8mpevk:~/hello_world$ ls  
CMSIS                build_debug.bat      config.cmake  
CMakeLists.txt       build_debug.sh       device  
MIMX8ML8             build_flash_debug.bat doc  
armgcc.cmake         build_flash_debug.sh drivers  
board                build_flash_release.bat evkmimx8mp  
build_all.bat        build_flash_release.sh flags.cmake  
build_all.sh         build_release.bat    mcux_config.cmake  
build_ddr_debug.bat  build_release.sh     remoteproc  
build_ddr_debug.sh   clean.bat           source  
build_ddr_release.bat clean.sh            startup  
build_ddr_release.sh component          utilities  
debix@imx8mpevk:~/hello_world$ ./build_all.sh  
-- TOOLCHAIN_DIR: /home/debix/tools/gcc-arm-none-eabi-9-2019-q4-major
```

- After compiling, enter the release directory

```
debix@imx8mpevk:~/hello_world$ cd release/  
debix@imx8mpevk:~/hello_world/release$ ls  
hello_world.bin  hello_world.elf  
debix@imx8mpevk:~/hello_world/release$
```

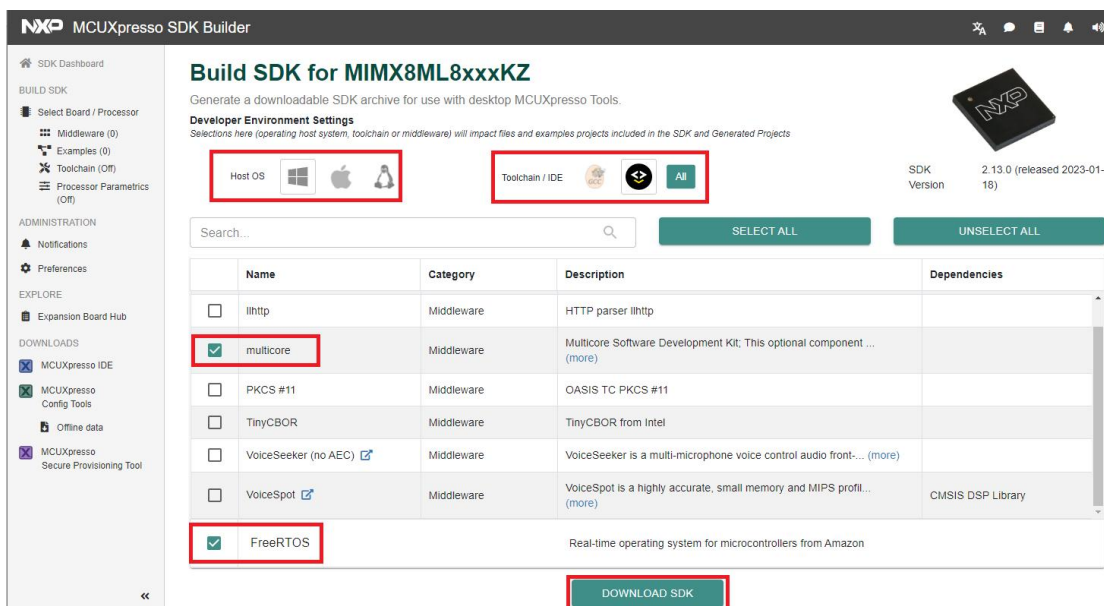
The compiler automatically generates `hello_world.bin` and `hello_world.elf` files.

You can run hello_world.bin on M7 and Linux on A53 by using the method in section 2.3. of "Parallel A53 and M7 cores".

4. Run M7 core with FreeRTOS

4.1. Download firmware

- Get the SDK package from NXP official website: <https://mcuxpresso.nxp.com/en/select>.
(the package named EVK-MIMX8MP-rpmsg_lite_str_echo_rtos_imxcm7.zip)
- Type **MIMX8ML8xxxKZ** in the search box of "Search for Hardware"
- Select **MIMX8ML8xxxKZ** processor
- Click **Build MCUXpresso SDK** button on the right side
- Select the Host OS, and select Toolchain / IDE
- Check **multicore** and **FreeRTOS**, and click **DOWNLOAD SDK**.



NXP MCUXpresso SDK Builder

Build SDK for MIMX8ML8xxxKZ
Generate a downloadable SDK archive for use with desktop MCUXpresso Tools.

Developer Environment Settings
Selections here (operating host system, toolchain or middleware) will impact files and examples projects included in the SDK and Generated Projects

Host OS: Windows, macOS, Linux
Toolchain / IDE: All
SDK Version: 2.13.0 (released 2023-01-18)

Search...

	Name	Category	Description	Dependencies
☐	ilhttp	Middleware	HTTP parser ilhttp	
☒	multicore	Middleware	Multicore Software Development Kit: This optional component ... (more)	
☐	PKCS #11	Middleware	OASIS TC PKCS #11	
☐	TinyCBOR	Middleware	TinyCBOR from Intel	
☐	VoiceSeeker (no AEC)	Middleware	VoiceSeeker is a multi-microphone voice control audio front-... (more)	
☐	VoiceSpot	Middleware	VoiceSpot is a highly accurate, small memory and MIPS profil... (more)	CMSIS DSP Library
☒	FreeRTOS		Real-time operating system for microcontrollers from Amazon	

DOWNLOAD SDK

- Select **Download package**



SDK_2.13.0_MIMX8ML8xxxKZ 2023-03-23

Windows 2.13.0 MIMX8ML8xxxKZ

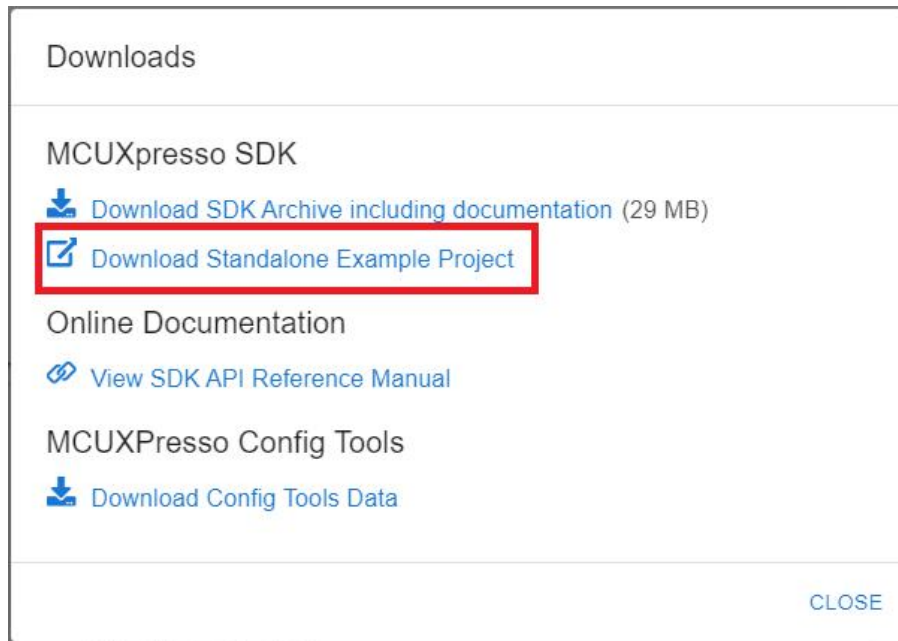
IAR Embedded Workbench for ARM

multicore, FreeRTOS

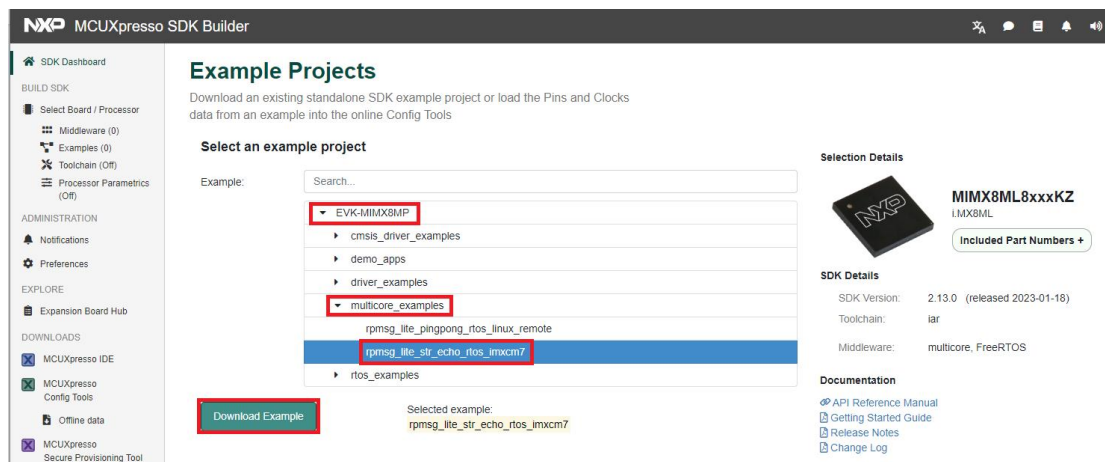
Add Package Description

Remove package
Rebuild package
Config Tools
Share package
Download package

- Select **Download Standalone Example Project**



- Select **rpmsg_lite_str_echo_rtos_imxcm7**, and click **Download Example** button



After downloading, you will get a **EVK-MIMX8MP-rpmsg_lite_str_echo_rtos_imxcm7.zip**, which is the project you will use.

Note: Select **freertos_hello**, download the package named **EVK-MIMX8MP-freertos_hello.zip**, which can also be used as a project.

4.2. Modify kernel source code and recompile the kernel

Modify the kernel source code and recompile the kernel in the same way as in section 2.1.

and section 2.2. of "Parallel A53 and M7 cores".

4.3. Compile the project

- Copy the compiled output `rpmsg_lite_str_echo_rtos.bin` to DEBIX, and during the uboot boot press any key to enter the uboot command line and type:

```
fatload mmc 1:1 0x48000000 rpmsg_lite_str_echo_rtos.bin;cp.b 0x48000000 0x7e0000
20000;
bootaux 0x7e0000
boot
```

```
u-boot=> fatload mmc 1:1 0x48000000 rpmsg_lite_str_echo_rtos.bin;cp.b 0x48000000
28032 bytes read in 5 ms (5.3 MiB/s)
u-boot=> bootaux 0x7e0000
## Starting auxiliary core stack = 0x20020000, pc = 0x00006465...
u-boot=> boot
starting USB...
```

M7 prints on serial port 4 as follows:

```
RPMSG String Echo FreeRTOS RTOS API Demo...
```

- After Linux boots normally, enter the command on the DEBUG serial port of the A53:

```
sudo su
```

```
modprobe imx_rpmsg_tty
```

```
Last login: Thu Mar 23 10:42:30 CST 2023 on ttymxc1
debix@imx8mpevk:~$
debix@imx8mpevk:~$ sudo su
[sudo] password for debix:
root@imx8mpevk:/home/debix# modprobe imx_rpmsg_tty
root@imx8mpevk:/home/debix#
```

The M7 terminal shows that the default message sent by the `rpmsg` driver on the A53:

```
RPMSG String Echo FreeRTOS RTOS API Demo...

Nameservice sent, ready for incoming messages...
Get Message From Master Side : "hello world!" [len : 12]
```

- Enter the command on the DEBUG serial port of the A53:

```
echo 'Test String..' > /dev/ttyRPMSG30
```

```
root@imx8mpevk:/home/debix# echo 'Test String..' > /dev/ttyRPMSG30
```

M7 receives the message and prints:

```
RPMSG String Echo FreeRTOS RTOS API Demo...

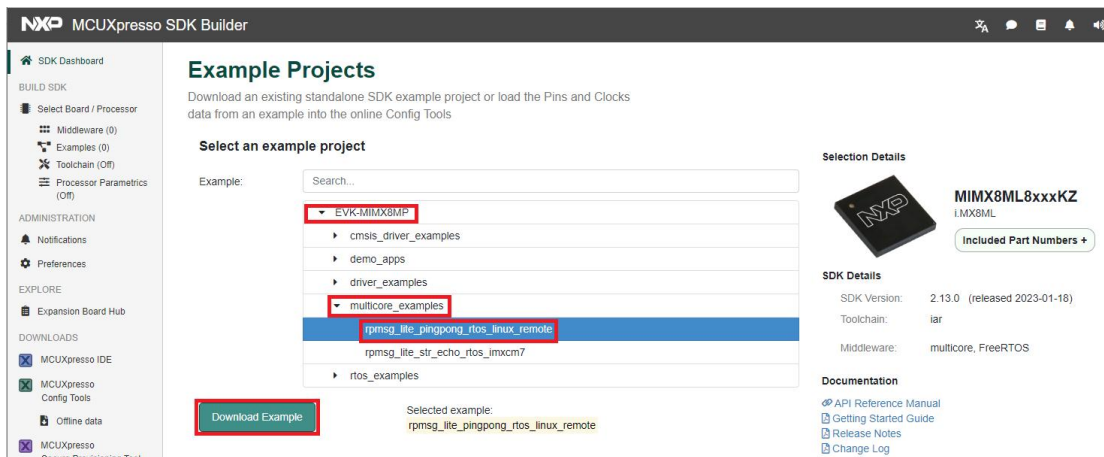
Nameservice sent, ready for incoming messages...
Get Message From Master Side : "hello world!" [len : 12]
Get Message From Master Side : "Test String.." [len : 13]
Get New Line From Master Side
```

5. Communication between M7 core and A53 core

5.1. Download firmware

The first few steps to download the firmware are the same as in section 4.1. (the package named EVK-MIMX8MP-rpmsg_lite_pingpong_rtos_linux_remote.zip).

Select `rpmsg_lite_pingpong_rtos_linux_remote` to download the example.



5.2. Modify kernel source code and recompile the kernel

Modify the kernel source code and recompile the kernel in the same way as in section 2.1. and section 2.2. of "Parallel A53 and M7 cores".

5.3. Compile the project

- Copy the compiled output `rpmsg_lite_pingpong_rtos_linux_remote.bin` to DEBIX, and during the uboot boot press any key to enter the uboot command line and type:

```
fatload mmc 1:1 0x48000000rpmsg_lite_pingpong_rtos_linux_remote.bin;cp.b 0x48000000
```

```
0x7e0000 20000;
```

```
bootaux 0x7e0000
```

```
boot
```

```
u-boot=> fatload mmc 1:1 0x48000000 rpmsg_lite_pingpong_rtos_linux_remote.bin;cp
.b 0x48000000 0x7e0000 20000;
28768 bytes read in 3 ms (9.1 MiB/s)
u-boot=> bootaux 0x7e0000
## Starting auxiliary core stack = 0x20020000, pc = 0x000066D9...
u-boot=> boot
starting USB...
```

M7 core serial port prints as follows:

```
RPMSG Ping-Pong FreeRTOS RTOS API Demo...
```

- After Linux boots normally, enter the command on the DEBUG serial port of the A53:

```
sudo su
```

```
modprobe imx_rpmsg_pingpong
```

```
debix@imx8mpevk:~$ sudo su
[sudo] password for debix:
root@imx8mpevk:/home/debix# modprobe imx_rpmsg_pingpong
```

M7 core prints as follows:

```
RPMSG Ping-Pong FreeRTOS RTOS API Demo...
RPMSG Share Base Addr is 0x55000000
Link is up!
Nameservice announce sent.
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
Waiting for ping...
Sending pong...
```

```
Waiting for ping...
Sending pong...
Ping pong done, deinitializing...
Looping forever...
```

A53 core prints as follows:

```
[ 70.247180] get 93 (src: 0x1e)
[ 70.250504] get 95 (src: 0x1e)
[ 70.253828] get 97 (src: 0x1e)
[ 70.257154] get 99 (src: 0x1e)
[ 70.260483] get 101 (src: 0x1e)
[ 70.260490] imx_rpmsg_pingpong virtio0.rpmsg-openamp-demo-channel.-1.30: good
bye!
```

The above proves that A53 and M7 send and receive msg to each other.